

なんとしても **PHP** のメモリを読む

PHP プロセスのメモリダンプが **3** 通りに増殖した話

reli / ext-rdump / php-memory-dump

sji (@sji_ch)

2026-07-10 第 3 回 仙台 PHP 勉強会 の開催中に vibe coding したスライドです

今日の話

「PHP プロセスのメモリダンプを取る」方法が、
制約と戦っているうちに 3 通りに増殖した話。

1. **reli** — FFI でシステムコールを呼び、外から別プロセスをダンプ
2. **ext-rdump** — C 拡張で中から自プロセスをダンプ
3. **php-memory-dump** — 純 **PHP** で中から自プロセスをダンプ

オチ: 「コレが動くなれば純 PHP でも実は可能なのでは？」 → 可能だった

前提: **reli** とは

<https://github.com/reliforp/reli-prof>

- PHP 製の PHP 用プロファイラ / メモリインスペクタ
- 対象プロセスに何も仕込まずに外からアタッチできる
 - サンプリングプロファイラ (`inspector:trace` , `top` , ...)
 - メモリダンプ + オフライン解析 (`inspector:memory:dump` → `analyze` → `report`)
- 対象は PHP 7.0+ (NTS/ZTS)、64bit Linux

reli のメモリダンプの仕組み(外から)

```
sudo php ./reli inspector:memory:dump --pid=<pid> --output=snapshot.rdump
```

1. 対象 PHP バイナリの **ELF** シンボルテーブルを解析して
 `executor_globals` / `compiler_globals` 等のリンク時アドレスを得る
2. `/proc/<pid>/maps` で分かるバイナリのベースアドレスを足して実アドレスに変換
3. **FFI** 経由で `ptrace(2)` / `process_vm_readv(2)` を呼んで
 対象プロセスのメモリ領域 (ZendMM チャンク、VMスタック、EG/CG...) を読む
4. `.rdump` ファイルに書き出し → 解析は別マシン・別時刻でオフラインに

ZendEngine のヒープ・オブジェクトストア・クラス表を歩くのは全部ダンプ後

これはこれで強い

- ターゲット側は無改造・無設定。動いてるプロセスにいきなり取り付ける
- メモリダンプは撮る一瞬だけ SIGSTOP で止めるのでオーバーヘッドはゼロではない。
ただしダンプ自体は速い (pagemap で非常駐ページを飛ばす等)。
場合によっては `gcore` にそこそこ勝つことも
- PHP 7.0 の古いプロセスも、ZTS (FrankenPHP) も対象にできる

でも.....

面倒ポイント①: **reli** を動かす側の **PHP**

reli 自体の実行要件:

- **PHP 8.4+**
- **FFI** 拡張が有効(+ PCNTL も)

→ 本番サーバの PHP が 7.4 だったら? FFI が無効ビルドだったら?

「プロファイルしたい対象」とは別に、
FFI の使える新しめの PHP を用意する必要がある

(Docker イメージも配ってるけど、それはそれで次のポイントが...)

面倒ポイント②: ptrace 権限

他人のプロセスのメモリを読むのは特権操作:

- 素で動かすなら **root** か `CAP_SYS_PTRACE`
(`setcap cap_sys_ptrace=eip $(which php)`)
- ハードニングされたホストでは Yama の
`/proc/sys/kernel/yama/ptrace_scope` が 2/3 だと **root** でもブロック
- コンテナからだと
`--cap-add=SYS_PTRACE --pid=host --security-opt apparmor=unconfined`
...

ローカルで手元検証する分にはまあいい。面倒になり得るのは本番・常時運用:

「ptrace できる特権を本番構成に入れて常駐させたい」は
インフラ・セキュリティ側への説得が要りがち

発想の転換

他人のメモリを読むから ptrace が要るのであって、
自分のメモリなら自分でコピーすればよいのでは？

プロセスは自分のアドレス空間を `memcpy` できる。当たり前。

- ELF シンボル解決 → 自分のバイナリを自分で読めばいい
- `/proc/<pid>/maps` → `/proc/self/maps`
- `process_vm_readv` → ただの `memcpy`

これを C 拡張として実装したのが **ext-rdump**

ext-rdump (中から・C 拡張)

<https://github.com/reliforp/ext-rdump>

```
rdump_dump( '/tmp/app.rdump' );
```

- 好きな瞬間に自プロセスのスナップショットを 1 関数で書き出す
- **ptrace** 不要・**FFI** 不要・**root** 不要
- PHP **7.0**～**8.5** (NTS/ZTS)、64bit Linux (x86-64 / arm64)
- 出力は reli と同じ **RDUMP** フォーマット
 - 解析はいつもの reli にそのまま食わせる (解析マシンにだけ reli があればいい)

```
reli inspector:memory:analyze app.rdump -f rmem -o app.rmem  
reli inspector:memory:report app.rmem
```

ext-rdump のおいしいところ

「中にいる」からこそできること:

- `memory_limit` 死の瞬間を自動ダンプ

```
rdump.oom_dump=/var/log/php-oom-%p.rdump
```

C の `zend_error_cb` フックなので、OOM エラー経路なら確実に捕まる

- シグナルハンドラから on-demand

```
pcntl_signal(SIGUSR2, fn () => rdump_dump('/tmp/sig.rdump'));
```

- ZTS で他スレッドが動いていても `rdump.safe_read=1`
(`/proc/self/mem` 経由の読み)でクラッシュせず読める

ところが今度は「拡張のインストール」が面倒

- プリビルドバイナリなし → コンパイラ + **php-dev** ヘッダが必要
- `pie install` / `pecl install` / `phpize && make && make install`
- `php.ini` に `extension=rdump.so`
- 本番環境に C 拡張を入れる、というそれなりの決断
(ビルド環境のない コンテナイメージだと特に)

`ptrace` の面倒は消えたが、導入の面倒が残った

とはいえ「PHP 処理系に `ptrace` 権限を渡す」よりは
通しやすい現場は多そう

(拡張を 1 個足すのは割と日常、特権付与はインフラ/セキュリティ審査になりがち)

ここで気づく

ext-rdump のやっていることを列挙してみると:

1. `/proc/self/maps` を読む → ただのテキストファイル読み
2. 自分の ELF バイナリのシンボル表をパース → ファイル読み + `unpack()`
3. メモリ領域を読んでファイルに書く → ...ここだけ **C** が必要?

そして Linux にはこういうファイルがある:

```
/proc/self/mem
```

自分のアドレス空間全体が「**seek** できるファイル」として見える

/proc/self/mem を PHP から読む

```
$fh = fopen('/proc/self/mem', 'rb');  
fseek($fh, $address);           // 仮想アドレスにシーク  
$bytes = fread($fh, $len);      // その番地のメモリが読める！
```

- 64bit ビルドの PHP なら `fseek` のオフセットも 64bit
→ スタックの `0x7fff...` みたいな高位アドレスにも届く
- ヒープも、mmap 領域も、ロードされたバイナリも全部これで読める

つまり必要な材料はぜんぶ **PHP** の標準機能で揃う。

拡張も FFI も要らない。

php-memory-dump (中から・純 PHP)

<https://github.com/reliforp/php-memory-dump>

```
composer require reliforp/php-memory-dump
```

```
use Reliforp\PhpMemoryDump\MemoryDumper;  
  
(new MemoryDumper())->dump('/tmp/app.rdump');
```

- 拡張なし・FFI なし・root なし。composer require だけ
- /proc/self/maps + ELF .dynsym パース + /proc/self/mem ストリーム読み
- 出力はやっぱり同じ **RDUMP** (magic **RDUMP** , format v3)
 - 解析はやっぱりいつもの reli

ちゃんと動くの? → 動く

素の Debian PHP 8.4 で self-dump → reli で解析すると
キャプチャ瞬間のコールスタックまで復元できる:

```
Call Stack at capture:  
#0 fread:-1  
#1 Reliforp\PhpMemoryDump\RegionReader::readAt:90  
#2 Reliforp\PhpMemoryDump\RegionReader::copyInto:75  
#3 Reliforp\PhpMemoryDump\RdumpWriter::writeRegion:129  
#4 Reliforp\PhpMemoryDump\MemoryDumper::dump:98  
#5 <main>:23
```

写っているのはダンパー自身が `fread` している瞬間。自撮りである

型ごとの内訳・オブジェクト一覧・リーク検出も、他のダンプと同様に出る

純 PHP ならではの注意点

- **NTS** 専用。ZTS はエンジングローバルが TLS の先にあり、
純 PHP からの解決は無理 → ZTS は ext-rdump か外からの reli で
- PHP コードが PHP のヒープを撮るので **stop-the-world** ではない
→ ダンプに自分の一時オブジェクトが写り込む(が、reli が歩ける程度には一貫)
- PHP バイナリに `executor_globals` 等のシンボルが必要
→ 幸い distro ビルドは strip されていても `.dynsym` にエクスポート済みなので OK
- 64bit little-endian Linux 限定(このへんは ext-rdump と同じ)

細かいこだわり

ヒープを撮る道具がヒープを荒らしては本末転倒なので:

- ELF は必要なスライス(ヘッダと `.dynsym / .dynstr`) だけ読む
- 領域は 256 KiB の固定バッファでストリーム書き
- → `dump()` の `emalloc` ピーク増分は **~0.6–1 MiB**、
新しい 2 MiB ZendMM チャンクを掴まない(ダンプサイズ非依存)

おかげで **OOM** 自動ダンプも純 **PHP** で書けた:

```
OomDumpHandler::register('/var/log/php-oom-%p-%t.rdump');
```

`memory_limit` を持ち上げ + 2 MiB 以上の緊急リザーブを先に解放してから撮る
(※ shutdown ハンドラ由来なのでベストエフォート。確実に捕るなら ext-rdump)

3 兄弟のまとめ

	reli (外から)	ext-rdump (中・C)	php-memory-dump (中・PHP)
導入	解析マシンに置くだけ	要ビルド (php-dev)	<code>composer require</code>
ptrace 権限	必要	不要	不要
FFI	必要 (実行側)	不要	不要
対象 PHP	7.0+ NTS/ZTS	7.0–8.5 NTS/ZTS	7.0+ NTS のみ
対象への仕込み	不要	拡張ロード	ライブラリ + 呼び出し
OOM の瞬間	watch/sidecar で	<code>zend_error_cb</code> で確実	ベストエフォート
スナップショット	SIGSTOP で一貫	同期・非 STW	同期・非 STW

解析はどれも同じ **reli**。フォーマットを RDUMP に揃えたのが効いた

まとめ

- 制約と戦うたびに新しい取り方が生えて、気づけば 3 通りに増殖
 - 外から (reli) は強いが **FFI** 入り **PHP** と **ptrace** 権限が面倒
 - 「自分のメモリなら ptrace 不要」→ **ext-rdump** (C 拡張)
 - 「コレが動いたら純 **PHP** でも実は可能なのでは？」→ **php-memory-dump**
- どれかが上位互換になったわけではなく、強みの違う 3 つの工具箱に
 - 仕込みゼロで今すぐ・ZTS も対象 → **reli**
 - OOM の瞬間を確実に捕る・ZTS の中から → **ext-rdump**
 - `composer require` だけの手軽さ → **php-memory-dump**
- カーネルが `/proc` でファイルとして見せてくれるものは、
PHP の `fopen` / `fseek` / `fread` でも読める。`/proc/self/mem` は偉い

おまけ: `/proc/self/mem`、よく見たら書けるやん

- `fopen('/proc/self/mem', 'r+b')` で開いて `fseek` → `fwrite` が通る
- しかもカーネルの書き込み経路は `FOLL_FORCE` でページ保護を無視する
→ 通常なら SEGV する読み取り専用ページや実行専用の `.text` にすら書ける
- つまり「読める」の隣に「書ける」が、権限チェックほぼザルで置いてある

読み取り = 今日のメモリダンプ。では書き込みは.....?

おまけ: **reli** の内部知識と合わせると変態的なことができる

reli 相当の「Zend の内部構造どこに何があるか」知識を足すと:

- 関数テーブルの `zend_internal_function` を見つけて **handler** ポインタを書き換え
- あるいは RX ページにマシンコードを流し込んで PHP から呼ばせる

→ **FFI** も **C** 拡張も無しに、純 PHP で
「syscall を直接叩く内部関数」を後付けできてしまう

```
// イメージ（本物は出しません）：  
// 1. /proc/self/maps + .dynsym で目的の番地を特定（今日の技術）  
// 2. /proc/self/mem に fwrite で機械語 or ポインタを書く  
// 3. PHP から呼ぶと...素の PHP なのに syscall
```

要素技術は今日のメモリダンプとほぼ同じ (maps + dynsym + self/mem)。

向きが read か write かの違いだけ

おまけ: これは「攻撃」ではない(大事)

- 前提は「もう自分で **PHP** を実行できている」こと
 - 新しく何かを乗っ取るわけではない。**RCE** でも権限昇格でもない
- できるのは自分のプロセス内で自分のコードを書き換える変態芸
- 唯一セキュリティに触れる含意:
 - `disable_functions` / FFI 無効 / `exec` 禁止 みたいな
 - PHP** レベルの制限を内側から迂回できる
 - 「PHP サンドボックス」は本来この程度には脆い、という話
- 実際のバイト列や手順は今回の話から脱線しすぎるので詳細は省略

メモリを撮れるということは、メモリに書けることのすぐ隣。

おぎょうぎのいい用途で止めておきましょう(自己責任)

リンク

- <https://github.com/reliforp/reli-prof>
- <https://github.com/reliforp/ext-rdump>
- <https://github.com/reliforp/php-memory-dump>

ご清聴ありがとうございました